Data Processing with Python

V FYIMP

Module 1

What is Python

Python is

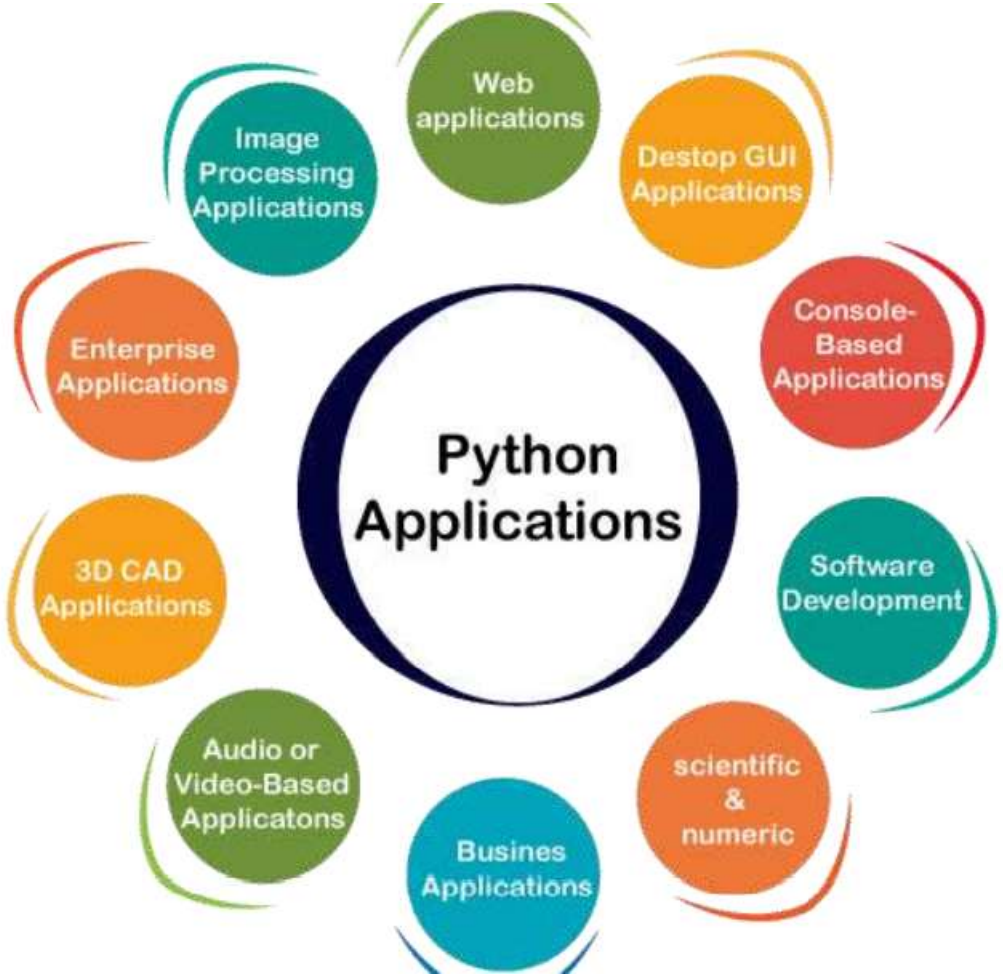
- **high-level**
- **interpreted**
- **object-oriented**
- **dynamic**
- **general-purpose**
- **programming language**

- Created by **Guido van Rossum** and first released in **1991**.
- It is known for its simple syntax, readability, and versatility, making it one of the most popular programming languages in the world.

Top Reason To Learn Python

Where is used Python Programming?

- Data Science
- Data Mining
- Desktop Applications
- Console-based Applications
- Mobile Applications
- Software Development
- Artificial Intelligence
- Web Applications
- Enterprise Applications
- 3D CAD Applications
- Machine Learning
- Computer Vision
- Image Processing Applications.
- Speech Recognitions



Why We Use Python(Features)

- **Easy to Learn:** Simple and readable syntax similar to English.
- **Interpreted Language:** Executes code line by line without requiring compilation.
- **Object-Oriented:** OOP is based on the concept of objects and classes.
- Supports classes, objects, inheritance, polymorphism, etc.
- **Platform Independent:** Python programs run on Windows, Linux, and macOS without modification.
- **Open Source:** Free to download, use, and modify.

Why We Use Python(Features) (contd.)

- **Dynamically Typed:** No need to declare variable types explicitly.
- **Large Standard Library:** Includes built-in modules for file handling, networking, mathematics, and more.
- **Extensible:** Can be integrated with languages like C and C++.
- Programmers can write performance-critical modules in C/C++ and use them in Python programs to improve efficiency.
- **Portable:** Programs can be executed on different operating systems.
- **Supports exception handling:** An exception is an event that can occur during program execution and can disrupt the normal flow of program.

Variables

- Reserved memory locations or it’s a name used to store values.
- It means that when you create a variable, you reserve some space in the memory.

```
counter = 100 # An integer assignment
miles = 1000.0 # A floating point
name = "John" # A string
print (counter)
print (miles)
print (name)
```

Multiple Assignment

- Python allows you to assign a single value to several variables simultaneously.

  a = b = c = 1

- You can also assign multiple objects to multiple variables

  a, b, c = 1, 2, "john”

Rules for Naming Variables

- Variable names must start with a **letter (A–Z, a–z)** or an **underscore (_)**.
- Variable names **cannot start with a number**.
- Variable names can contain **letters, digits (0–9), and underscores (_)**.
- **Spaces are not allowed** in variable names.
- **Special characters** such as @, #, $, %, &, ! are **not allowed**.
- Variable names are **case-sensitive** (age, Age, and AGE are different variables).
- Variable names **cannot be Python keywords** such as if, for, class, while, etc.

Valid Variable Names

- name = "John“
- _age = 25
- student1 = "Alice"
- total_marks = 450
- price2 = 99.99

- Do **not** need to declare the data type of a variable.
- Python automatically determines the type when a value is assigned.
- A variable can hold different types of values at different times.
- **Example**
  - x = 10
  - print(x)
  - x = "Python"
  - print(x)
  - x = 3.14
  - print(x)

Output:

10

Python

3.14

  - Here the variable x changes from integer(int),String(str),float(float)
  - variable types are determined during program execution (runtime) rather than before execution (compile time).
  - This is possible because Python is a **dynamic language**.

Operators and Operands in Python

- An **operator** is a **symbol** that performs an operation on one or more values.
- **Examples:** +, -, *, /, %, ==
- An **operand** is the **value or variable** on which an operator performs an operation.
- a = 10
- b = 5
- c = a + b

Operators

Python language supports the following types of operators-

- Arithmetic Operators
- Comparison (Relational) Operators
- Assignment Operators
- Logical Operators
- Bitwise Operators
- Membership Operators
- Identity Operators

1. Arithmetic Operators

- Used to perform mathematical operations.
- Assume a=10 ,b=21,then

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	a + b = 31
- Subtraction	Subtracts right hand operand from left hand operand.	a – b = -11
* Multiplication	Multiplies values on either side of the operator	a * b = 210
/ Division	Divides hand operand by right hand operand	b / a = 2.1
% Modulus	Divides left hand operand by right hand operand and returns remainder	b % a = 1
** Exponent	Performs exponential (power) calculation on operators	a**b =10 to the power 21
// Floor Division	- The division of operands where the result is the quotient in which the digits after the decimal point are removed.	21//10 = 2 and 21.0//10.0.0 = 2.0

2. Comparison Operators/ Relational Operators

- These operators compare the values on either side of them and decide the relation among them.
- Assume a =10 , b=20, then

Operator<	Description	Example
==	If the values of two operands are equal, then the condition become true	(a==b) is not true
!=	If values of two operands are not equal, then condition become true	(a!=b) is true
>	If the value of the left operand is greater than the value of the right operand, then condition become true	(a>b) is not true
<	If the value of the left operand is less than the value of the right operand, then condition become true	(a<b)is true
>=	If the value of the left operand is greater than or equal to the value of the right operand, then condition become true	(a>=b)is not true
<=	If the value of the left operand is less than or equal to the value of the right operand, then condition become true	(a<=b)is true

3. Assignment Operators

- Used to assign values to variables.

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

4. Logical Operators

- Used to combine multiple conditions.

Operator	Meaning	Example
`and`	True if both conditions are true	`a > 5 and b < 10`
`or`	True if at least one condition is true	`a > 5 or b > 10`
`not`	Reverses the result	`not(a > 5)`

5. Bitwise Operators

- Operate on the binary representation of integers.

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

6.Membership Operators

- Used to check whether a value exists in a sequence.

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

```
fruits = ["apple", "banana", "orange"]

print("apple" in fruits)
print("grape" not in fruits)
```

7.Identity Operators

- Used to compare whether two variables refer to the same object.

Operator	Description	Example
is	Returns true if both variables are the same object	x is y
is not	Returns true if both variables are not the same object	x is not y

```
a = [1, 2]
b = a
c = [1, 2]
print(a is b)
print(a is c)
```

Expressions in Python

- A **Python expression** is a combination of **variables, constants, operators, and function calls** that is evaluated to produce a single value.

- a = 10
- b = 5
- result = a + b
- print(result)

Types of Expressions

- Arithmetic Expressions
- Relational (Comparison) Expressions
- Logical Expressions
- Assignment Expressions
- Bitwise Expressions
- Membership Expressions
- Identity Expressions

1.Arithmetic Expressions

- Perform mathematical calculations.
- Python uses the arithmetic operators.

Operator	Example	Result
+	10 + 5	15
-	10 - 5	5
*	10 * 5	50
/	10 / 5	2.0
%	10 % 3	1
//	10 // 3	3
**	2 ** 3	8

2.Relational (Comparison) Expressions

- Compare two values.
- Python uses the relational operators.

Expression	Output
10 > 5	True
10 < 5	False
10 == 10	True
10 != 5	True

3.Logical Expressions

- Combine two or more conditions.
- Python uses the logical operators.
- a = 10 ,b = 5 then

Operator	Example
and	a > 5 and b < 10
or	a > 5 or b > 10
not	not(a > 5)

4.Assignment Expressions

- Assign values to variables.

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

5.Bitwise Expressions

- Operate on binary values.
- a = 5 b = 3 print(a & b)

Operator	Meaning	Expression
&	AND	a&b
|	OR	a|b
^	XOR	a^b
~	NOT	a~b
<<	Left Shift	a<<1
>>	Right Shift	b>>1

6.Membership Expressions

- Check whether a value exists in a sequence.
- Python uses the in and not in membership operators.

```
fruits = ["Apple", "Banana", "Orange"]


print("Apple" in fruits)
print("Grapes" not in fruits)
```

7.Identity Expressions

- Check whether two variables refer to the same object.
- Python uses the is and is not identity operators.

```
a = [1, 2]
b = a
c = [1, 2]


print(a is b)
print(a is not c)
```

Operator Precedence

- When an expression contains multiple operators, Python evaluates them according to **operator precedence**.
- **Operator precedence** determines the order in which operators are evaluated in an expression.
- Operators with **higher precedence** are evaluated before operators with **lower precedence**

Precedence and Associativity of operators

Operator
()
**
+x, -x, ~x
/, //, *, %
+, -
<<, >>
&
^
|
==,<, >, <=, >= , !=
not
and
or



Importing Libraries in Python

What is a Library?

- A **library** in Python is a collection of pre-written modules, functions, and classes that provide reusable code to perform specific tasks.
- Libraries help programmers avoid writing code from scratch and make development faster and easier.

Examples

- **math** – Mathematical operations
- **random** – Random number generation
- **numpy** – Numerical computing
- **pandas** – Data analysis
- **matplotlib** – Data visualization

Why Do We Import Libraries?

Libraries are imported to:

- Reuse existing code.
- Perform complex tasks easily.
- Reduce development time.
- Improve code readability and efficiency.

Types of Python Library

- Python libraries are divided into two main types:

1. Built-in or Standard Python Library

- It is a collection of modules that come bundled with every Python installation.
- we don’t need to install anything separately.
- Most of these modules are written in C for better performance.

Examples of built-in libraries:

- **math:** Mathematical operations
- **os:** Interact with the operating system
- **datetime:** Date and time operations
- **random:** Generate random numbers
- **json:** Handles JSON data encoding and decoding.

2. External or Third-party Python Libraries

- External (third-party) libraries are not included with Python by default.
- Need to be installed separately.

Examples of Third-party libraries:

1.NumPy:

- It, short for Numerical Python.
- It is the core library for numerical and scientific computing in Python.
- It provides powerful tools for creating and manipulating arrays, matrices and mathematical operations and statistics.

2.Pandas:

- It is a data analysis and manipulation library **and working with structured data** such as tables and CSV files.
- It introduces data structures like DataFrame and Series that make it easy to handle structured data efficiently.

3.Matplotlib:

- It is a data visualization library.
- That helps you create a wide variety of static, animated and interactive plots.
- It supports charts such as bar graphs, line charts, histograms and scatter plots.

Using Libraries in Python Programs

- To use any library, it first needs to be imported into your Python program using the import statement.
- Once imported, you can directly call the functions or methods defined inside that library.
- You can import libraries in three main ways:

1.Import the entire library:

import library_name

Ex:- import math

Example Program:

```
import math
A = 16
print(math.sqrt(A))
```

- Here, the complete math library is imported and we use math.sqrt() to calculate the square root of 16.
- Since the full library is imported, we must prefix the function with the library name (math.).

2.Import a specific function or class:

from library_name import function_name

Ex:- from math import sqrt

Example Program:

```
from math import factorial

# Input from the user
num = int(input("Enter a number: "))

# Calculate factorial
result = factorial(num)

# Display the result
print("The factorial of", num, "is", result)
```

3.Import Multiple Functions:

from library_name import function_name, function_name

Ex:- from math import sqrt,factorial

- Example Program:

```
from numpy import array, mean
a = array([10, 20, 30, 40, 50])
print(mean(a))
```

•Here, only the array() and mean() functions are imported from the NumPy library.

•array() is used to create a NumPy array from a list and mean() calculates the average value of all elements in the array.

•Since these functions are imported directly, we don’t need to use the numpy. prefix each time we call them.

4.Import a library with an alias:

import library_name as alias

Ex:- import pandas as pd

- An **alias** gives a shorter name to a library.
- Example Program:

```
import numpy as np


arr = np.array([10,20,30])
print(arr)
```

import math and **from math import ***

Using from math import *

- **Syntax:**

  from math import *

- **Example:**

```
from math import *
print(sqrt(25))
print(factorial(5))
print(pi)
```

- All functions and constants from the math module are imported.
- You can call sqrt(), factorial(), and pi directly without writing math..

import math

- Imports the entire math module.
- Functions are accessed using the module name (math.function_name).
- Avoids name conflicts with functions from other modules.
- Recommended for most programs.

from math import *

- Imports all functions and constants from the math module.
- Functions can be called directly without the module name.
- May cause name conflicts if different modules have functions with the same name.
- Generally discouraged in larger programs.

Why is **from math import *** not recommended?

- Suppose another module also has a function named sqrt().
- If you import everything from both modules, Python won't know which sqrt() you mean, leading to **name conflicts.**
- For example:

```
from math import *
from cmath import *


print(sqrt(25))
```

- Here, it may not be obvious whether sqrt() is from the math module or the cmath module.
- Using import math avoids this problem:

```
import math
import cmath

print(math.sqrt(25))
print(cmath.sqrt(25))
```

- Now it is clear which function is being used.

Installing Third-Party Libraries

- Use **pip** to install libraries.
- **syntax**
- pip install library_name
- Ex:-
  - pip install numpy
  - pip install pandas
  - pip install matplotlib

Data Types

Python has five standard data types-

- Numbers
- String
- List
- Tuple
- Dictionary

Numbers

- Number data types store numeric values.
- Number objects are created when you assign a value to them.
- For example

  var1 = 1; var2 = 10

- Python supports three different numerical types
  - int (signed integers)
  - float (floating point real values)
  - complex (complex numbers)

Strings

- Continuous set of characters represented in the quotation marks.
- Python allows either pair of single or double quotes.
- Subsets of strings can be taken using the slice operator ([ ] and [:]) with indexes starting at 0 in the beginning of the string
- The plus (+) sign is the string concatenation operator and the asterisk (*) is the repetition operator.

Strings

For example-

```
str = 'Hello World!'
print (str)          # Prints complete string – Hello World!
print (str[0])       # Prints first character of the string - H
print (str[2:5]) # Prints characters starting from 3rd to 5th- llo
print (str[2:])    # Prints string starting from 3rd character –llo World!
print (str * 2)    # Prints string two times –Hello World! Hello World!
print (str + "TEST") # Prints concatenated string- Hello World!Test
```

Lists

- Lists are the most versatile of Python's compound data types.
- A list contains items separated by commas and enclosed within square brackets ([]).
- To some extent, lists are similar to arrays in C.
- One of the differences between them is that all the items belonging to a list can be of different data type.
- The values stored in a list can be accessed using the slice operator ([ ] and [:]) with indexes starting at 0 in the beginning of the list and working their way to end -1.
- The plus (+) sign is the list concatenation operator, and the asterisk (*) is the repetition operator.

Lists

For example

```
list = [ 'abcd', 786 , 2.23, 'john', 70.2 ]
tinylist = [123, 'john']
print (list)          # Prints complete list
print (list[0])       # Prints first element of the list
print (list[1:3])     #Prints elements starting from 2nd till 3rd
print (list[2:])      # Prints elements starting from 3rd element
print (tinylist * 2)  # Prints list two times
print (list + tinylist) # Prints concatenated lists
```

Output

```
['abcd', 786, 2.23, 'john', 70.2]
abcd
[786, 2.23]
[2.23, 'john', 70.2]
[123, 'john', 123, 'john']
['abcd', 786, 2.23, 'john', 70.2, 123, 'john']
```

Tuples

- Similar to the list.
- Consists of a number of values separated by commas.
- Enclosed within parenthesis.
- The main difference between lists and tuples is-
- Lists are enclosed in brackets ([ ]) and their elements and size can be changed
- Tuples are enclosed in parentheses (( )) and cannot be updated.
- Tuples can be thought of as read-only lists.

Tuples

For example-

```
tuple = ( 'abcd', 786 , 2.23, 'john', 70.2 )
tinytuple = (123, 'john')
print (tuple)       # Prints complete tuple
print (tuple[0]) # Prints first element of the tuple
print (tuple[1:3]) # Prints elements starting from 2nd till 3rd
print (tuple[2:]) # Prints elements starting from 3rd element
print (tinytuple * 2) # Prints tuple two times
print (tuple + tinytuple) # Prints concatenated tuple
tuple = ( 'abcd', 786 , 2.23, 'john', 70.2 )
list = [ 'abcd', 786 , 2.23, 'john', 70.2 ]
tuple[2] = 1000 # Invalid syntax with tuple
list[2] = 1000 # Valid syntax with list
```

Output

```
('abcd', 786, 2.23, 'john', 70.2)
abcd
(786, 2.23)
(2.23, 'john', 70.2)
(123, 'john', 123, 'john')
('abcd', 786, 2.23, 'john', 70.2, 123, 'john')
```

Set

- Sets are used to store multiple items in a single variable.
- Set is one of 4 built-in data types in Python used to store collections of data, the other 3 are List, Tuple, and Dictionary, all with different qualities and usage.
- A set is a collection which is *unordered*, *unchangeable*, and *unindexed*.
- Sets are written with curly brackets.

Example:

```
thisset = {"apple", "banana", "cherry"}
print(thisset)
```

Dictionary

- Dictionaries are used to store data values in key:value pairs.
- A dictionary is a collection which is ordered, changeable and does not allow duplicates.
- Dictionaries are written with curly brackets, and have keys and values:
- Dictionaries are enclosed by curly braces ({ }) and values can be assigned and accessed using square braces ([]).

Example:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict)
```

Dictionary

- For example-

```
dict = {}
dict['one'] = "This is one"
dict[2] = "This is two"
tinydict = {'name': 'john','code':6734, 'dept': 'sales'}
print (dict['one']) # Prints value for 'one' key
print (dict[2]) # Prints value for 2 key
print (tinydict) # Prints complete dictionary
print (tinydict.keys()) # Prints all the keys
print (tinydict.values()) # Prints all the values
```

Output

```
This is one
This is two
{'name': 'john', 'code': 6734, 'dept': 'sales'}
dict_keys(['name', 'code', 'dept'])
dict_values(['john', 6734, 'sales'])
```

Mutable and immutable objects in Python

- There are two types of objects in python i.e.
- **Mutable** and **Immutable objects**.
- Whenever an object is instantiated, it is assigned a unique object id.
- The type of the object is defined at the runtime and it can’t be changed afterwards.
- However, it’s state can be changed if it is a mutable object.
- **Immutable Objects :** These are of in-built types like **int, float, bool, string, unicode, tuple**. In simple words, an immutable object can’t be changed after it is created.
- **Mutable Objects :** These are of type list, dict, set .

- In Python, every value is stored as an **object**.
- Objects are classified into two types based on whether their contents can be changed after creation:

- **Mutable Objects** – Can be modified after they are created.
- **Immutable Objects** – Cannot be modified after they are created.

Mutable Objects

- A mutable object's contents can be modified without changing its identity(memory address) or without creating a new object.

- **Examples**
- list
- dict
- set

- **Example 1: List**

```
numbers = [10, 20, 30]
print(numbers)
numbers.append(40)
print(numbers)
```

Output

```
[10, 20, 30]
[10, 20, 30, 40]
```

- The list numbers is created.
- append(40) adds a new element to the same list.
- The original object is modified.

Example 2: Dictionary

```
student = {"name": "John", "age": 20}


student["age"] = 21
print(student)
```

Output

{'name': 'John', 'age': 21}

The dictionary is modified without creating a new object.

- If you assign a value to a key that isn't already in the dictionary, Python creates a **new key-value pair**.

Example 3:

```
student = {"name": "John", "age": 20}
student["city"] = "New York"
print(student)
```

Output

{'name': 'John', 'age': 20, 'city': 'New York'}

Example 4:

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
print(b)
```

Output

```
[1, 2, 3, 4]
[1, 2, 3, 4]
```

- a and b refer to the same list.
- Modifying b also changes a.

Immutable Objects

- An **immutable object** cannot be changed after it is created.
- If you appear to modify it, Python actually creates a **new object**.
- **Common Immutable Data Types**
- int
- float
- str
- tuple

Example 1: Integer

```
x = 10
print(x)

x = x + 5
print(x)
```

Output

```
10
15
```

- Initially x refers to the integer 10.
- x = x + 5 creates a **new integer object** (15).
- The original integer (10) is not modified.

Example 2: String

```
name = "Python"
name = name + " Programming"
print(name)
```

Output

```
Python Programming
```

- Strings cannot be modified.
- Python creates a new string "Python Programming".

Example 3:

```
a = "Hello"
b = a
b = b + " World"
print(a)
print(b)
```

Output

```
Hello
Hello World
```

- b now refers to a new string.
- The original string assigned to ‘a’ remains unchanged.

id() Function

- The id() function returns the identity (memory address) of an object.
- This identifier is usually the **memory address** where the object is stored.

Example 1: List Object(mutable)

```
numbers = [10, 20, 30]
print("Before:",numbers)
print(id(numbers))
numbers.append(40)
print("After:",numbers)
print(id(numbers))
```

Output

Before:[10, 20, 30]

140722903876800

After:[10,20,30,40]

140722903876800

Example 2: Dictionary Object(mutable)

```
student = {"name": "John", "age": 20}
print("Before:", student)
print("ID:", id(student))
student["age"] = 21
print("After:", student)
print("ID:", id(student))
```

Output

Before: {'name': 'John', 'age': 20}

ID: 140234567890112

After: {'name': 'John', 'age': 21}

ID: 140234567890112

Example 3: Integer (Immutable)

```
x = 10
print("Value of x:", x)
print("ID of x:", id(x))
x = x + 5
print("\nAfter modification:")
print("Value of x:", x)
print("ID of x:", id(x))
```

Output

```
Value of x: 10
ID of x: 140716218735816
After modification:
Value of x: 15
ID of x: 140716218735976
```

Reading Input

- Python provides some built-in functions to perform both input and output operations.
- **Python provides two inbuilt functions to read the input from the keyboard.**

  raw_input()

  input()

- **raw_input():** This function reads only one line from the standard input and returns it as a String-(Python 2)

- **input():** The input() function first takes the input from the user and then evaluates the expression, which means python automatically identifies whether we entered a string or a number or list-(Python2)

- But in Python 3 the raw_input() function was removed and renamed to input().
- input() always returns a **string**.

Example 1: Reading a String

```
name = input("Enter your name: ")
print("Hello,", name)
```

Output

Enter your name: Ram

Hello, Ram

Example 2: Reading an Integer

```
age = int(input("Enter your age: "))
print("Your age is", age)
```

Output

Enter your age: 20

Your age is 20

Example 3: Reading Two Numbers in One Line

```
a, b = map(int, input("Enter two numbers: ").split())
print("First Number:", a)
print("Second Number:", b)
```

Output

Enter two numbers: 10 20

First Number: 10

Second Number: 20

- input() reads the entire line as a string.
- .split() separates the input into individual strings using spaces.
- The split() method divides the string into a list using spaces as the default separator.
- map(int, ...) converts each string into an integer.
- map() applies int() to **each element**:
- The values are assigned to a and b.

- The one-line statement

```
a, b = map(int, input("Enter two numbers: ").split())
```

- is equivalent to:

```
text = input("Enter two numbers: ")
parts = text.split()
num1 = int(parts[0])
num2 = int(parts[1])
a = num1
b = num2
print(a)
print(b)
```

Print Output

- In order to print the output, python provides us with a built-in function called print().

Example:

print(“Hello Python”)

Output:

Hello Python

Difference between Method and Function in Python

Function

- A function is a block of code to carry out a specific task, will contain its own scope and is called by name.
- All functions may contain zero(no) arguments or more than one arguments.

Difference between Method and Function in Python

Method

- A method in python is somewhat similar to a function, except it is associated with object/classes.
- Methods in python are very similar to functions except for two major differences.
- The method is implicitly used for an object for which it is called.
- The method is accessible to data that is contained within the class.

List

- Creating a list
- Basic List operations
- Indexing and slicing in Lists
- Built-in functions used on lists
- List methods
- The del statement

List

1.Creating a list

- Lists are used to store multiple items in a single variable.

```
thislist = ["apple", "banana", "cherry"]
print(thislist)
```

- We can **update** lists by using slice[] on the LHS of the assignment operator

Eg:

```
list=['bcd',147,2.43,'Tom']
print("Item at index 2=",list[2])
list[2]=500
print("Item at index 2=",list[2])
```

Output

2.43

500

List

- To remove an item from a list
  - del statement
  - remove() method

```
list=['abcd',147,2.43,'Tom',74.9]
print(list)
del list[2]
print("list after deletion:", list)
```

Output

['abcd',147,2.43,'Tom',74.9]

list after deletion:['abcd',147,'Tom',74.9]

- The del keyword can also used to delete the list completely
- Deletes an element at a specified index.

```
thislist = ["apple", "banana", "cherry"]
del thislist
```

- **remove()function**
- Removes the first occurrence of an element.

```
thislist = ["apple", "banana", "cherry"]
thislist.remove("banana")
print(thislist)
```

2. Indexing and Negative Indexing

- Python lists are **zero-indexed**, meaning the first element starts at position 0.
- **Positive Indexing**: Moves from left to right, starting at 0.
- **Negative Indexing**: Moves from right to left, starting at -1 (accesses elements from the end).

```
letters = ["a", "b", "c", "d"]

print(letters[0])   # Output: 'a' (First element)
print(letters[2])   # Output: 'c' (Third element)

print(letters[-1])  # Output: 'd' (Last element)
print(letters[-3])  # Output: 'b' (Third from the last)
```

3. List Slicing

- Slicing allows you to extract a specific portion (sublist) of a list.
- **Slicing** is used to obtain a portion of a list.
  Syntax: list[start:stop:step]
- start: The starting index (inclusive). Defaults to 0.
- stop: The ending index (exclusive). Defaults to end of list.
- step: The stride or increment value. Defaults to 1.

```
numbers = [0, 10, 20, 30, 40, 50, 60]
print(numbers[1:5])     # Output: [10, 20, 30, 40]          # Extract index 1 to 4 (5 is excluded)

print(numbers[:3])      # Output: [0, 10, 20]               # Get everything from the beginning up to index 3

print(numbers[::2])     # Output: [0, 20, 40, 60]           # Get elements with a step of 2

print(numbers[::-1])    # Output: [60, 50, 40, 30, 20, 10, 0] # Reverse a list using a negative step
```

Updating Elements in the list

- Since lists are mutable, elements can be updated by assigning new values using their index.

```
a = [10, 20, 30, 40, 50]
a[1] = 25
print(a)
```

Output

[10, 25, 30, 40, 50]

Built-in list functions

1. **len(list)** – Gives the total length of list
2. **max(list)-** Returns item from list with maximum value
3. **min(list)-** Returns item from list with minimum value
4. **list(seq)-** Returns a tuple into a list
5. **map(aFunction,aSequence)** – Apply an operation to each item and collect result.

Example

```
list1=[1200,147,2.43,1.12]
list2=[213,100,289]
print(list1)
print(list2)
print(len(list1))
print("Maximum value in list1 is ",max(list))
print("Maximum value in list2 is ",min(list))
```

Output

[1200,147,2.43,1.12]
[1200,147,2.43,1.12]
4
Maximum value in the list1 is 1200
Minimum value in the list2 is 100

Example of list() and map() function

```
tuple = ('abcd',147,2.43,'Tom')
print("List:",list(tuple))
str=input("Enter a list(space separated):")
lis=list(map(int,str.split()))
print(lis)
```

Output

List: ['abcd',147,2.43,'Tom']

Enter a list (space separated) : 5 6 8 9

[5,6,8,9]

In this example a string is read from the keyboard and each item is converted into int using map() function

Built-in list methods

1. **list.append(obj)** –Append an object obj passed to the existing list

```
list = ['abcd',147,2.43,'Tom']
print("Old list before append:", list)
list.append(100)
print("New list after append:",list)
```

Output

Old list before append: ['abcd',147,2.43,'Tom']

New list after append: ['abcd',147,2.43,'Tom',100]

2. list.count(obj) –Returns how many times the object obj appears in a list

```
list = [‘abcd’,147,2.43,’Tom’]
print(“The number of times”,147,”appears in the
list=”,list.count(147))
```

Output

The number of times 147 appears in the list = 1

3. **list.remove(obj)** – Removes the first occurrence of an element.

```
list1 = ['abcd',147,2.43,'Tom']
list.remove('Tom')
print(list1)
```

Output

```
['abcd',147,2.43]
```

4. **list.index(obj)** – Returns index of the object obj if found

```
print(list1.index(2.43))
```

Output

2

5. list.extend(seq)- Appends the contents in a sequence passed to a list

```
list1 = ['abcd',147,2.43,'Tom']
list2 = ['def',100]
list1.extend(list2)
print(list1)
```

Output

```
['abcd',147,2.43,'Tom','def',100]
```

6. list.reverse() – Reverses objects in a list

```
list1.reverse()
print(list1)
```

Output

```
['Tom',2.43,147,'abcd']
```

7. list.insert(index,obj)- Returns a list with object obj inserted at the given index

```
list1 = [‘abcd’,147,2.43,’Tom’]
list1.insert(2,222)
print(“List after insertion”,list1)
```

Output

```
[‘abcd’,147,222,2.43,’Tom’]
```

8. **list.sort([Key=None,Reverse=False])** – Sort the items in a list and returns the list, If a function is provided, it will compare using the function provided

```
list1=[890,147,2.43,100]

print("List before
sorting:",list1)#[890,147,2.43,100]

list1.sort()

print("List after sorting in ascending
order:",list1)#[2.43,100,147,890]

list1.sort(reverse=True)

print("List after sorting in descending
order:",list1)#[890,147,100,2.43]
```

9.list.pop([index]) – removes or returns the last object obj from a list. we can pop out any item using index

```
list1=['abcd',147,2.43,'Tom']
list1.pop(-1)
print("list after poping:",list1)
```

Output

```
List after poping: ['abcd',147,2.43]
```

10. list.clear() – Removes all items from a list

```
list1.clear()
```

11. list.copy() – Returns a copy of the list

```
list2=list1.copy()
```

Using List as Stack

- List can be used as stack(Last IN First OUT)
- To add an item to the top of stack – ***append()***
- To retrieve an item from top –***pop()***

```
stack = [10,20,30,40,50]
stack.append(60)
print("stack after appending:",stack)
stack.pop()
print("Stack after poping:",stack)
```

Output

Stack after appending:[10,20,30,40,50,60]

Stack after poping:[10,20,30,40,50]

Using List as Queue

- List can be used as **Queue data structure(FIFO)**
- Python provide a module called collections in which a method called deque is designed to have append and pop operations from both ends

```
from collections import deque
queue=deque(["apple","orange","pear"])
queue.append("cherry")#cherry added to right end
queue.append("grapes")# grapes added to right end
queue.popleft()  # first element from left side is removed
queu.popleft()   # first element in the left side removed
print(queue)
```

Output

deque(['pear','cherry','grapes'])